

Java DSA I

학습 체크리스트 (1/2)

- ☐ 가능한 모든 경우의 수를 대입하여 정답을 보장하는 완전탐색(Brute Force)의 원리 이해
- ☐ 탐색 대상 문제를 적절히 구조화하고 경우의 수를 안전하게 제어하는 설계력 배양
- ☐ 정렬된 데이터에서 범위를 반씩 줄여나가는 이분탐색(Binary Search)의 작동 조건 숙지

학습 체크리스트 (2/2)

☐ 최적화 문제를 결정 문제로 전환하여 경계를 조여가는 매개 변수 탐색(Parametric Search) 기법 습득

☐ Java String의 불변(Immutable) 특성과 이로 인한 힙 메모리 오버헤드 이해

☐ 가변 버퍼를 지원하는 StringBuilder를 활용한 효율적인 문자열 조작 및 최적화 기법 습득

완전탐색

완전탐색의 정의

완전탐색 (Exhaustive Search / Brute Force)

문제 해결을 위해 가능한 모든 경우의 수를 하나도 빠짐없이 대입하여 해를 구하는 가장 직관적이고 무식한(Brute-force) 탐색 기법

- **안전성:** 복잡한 최적화 아이디어가 즉각 떠오르지 않을 때 정답을 100% 보장하는 가장 확실한 카드
- **도입 조건:** 전체 데이터의 입력 한도가 충분히 작아, 모든 경우를 돌아도 제한시간을 초과하지 않을 때 최우선 고려
- **정렬과의 결합:** 데이터 정렬은 완전탐색의 예비 단계(예: 이분 탐색 선결 조건)나 탐색 결과의 최종 가공 과정에서 필수적으로 결합됨

문제 1: 최소직사각형 (Lv. 1)

- **회전 조건 고려:** 명함을 돌려 놓혀 수납할 수 있으므로, 가로와 세로의 절대적 명칭에 갇히지 말고 유연하게 정렬하여 배치해야 함
- **규격의 표준화:** 각 명함 크기 중에서 더 긴 쪽을 무조건 한쪽(예: 가로)으로 통일하고, 더 짧은 쪽을 다른 한쪽(예: 세로)으로 통일
- **최소 수용 조건:** 지갑이 모든 명함을 수납하려면 가로 그룹의 최대값(`maxW`)과 세로 그룹의 최대값(`maxH`)이 각각 지갑의 최소 크기로 설정되어야 함
- **시간 복잡도:** 전체 명함 크기 배열을 단일 루프로 1회 순회하므로 $O(N)$ 의 선형 시간에 해결 가능

명함 회전 및 최소 지갑 크기 계산

```
1  for (int[] size : sizes) {  
2      // Math.max/min을 이용해 긴 쪽과 짧은 쪽 분리  
3      int w = Math.max(size[0], size[1]);  
4      int h = Math.min(size[0], size[1]);  
5  
6      // 각 그룹의 최대값(maxW, maxH)을 Math.max로 누적 갱신  
7  }
```

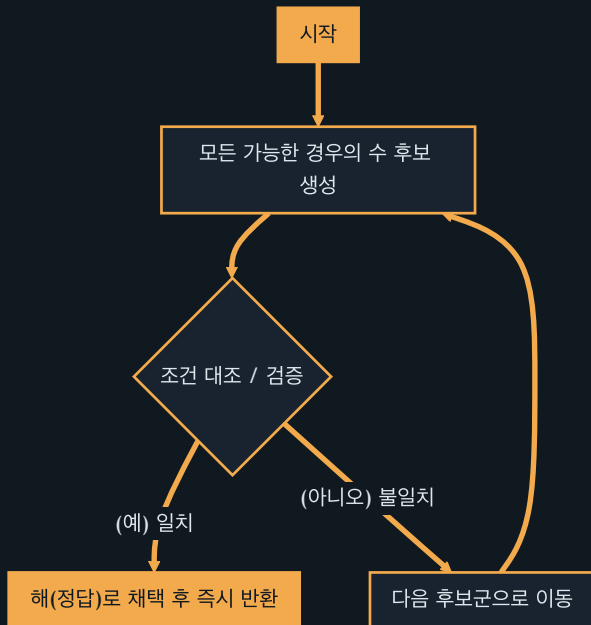
문제 2: 카펫 (Lv. 2)

- 수학적 조건 도출: 가로 w , 세로 h 인 카펫에서 테두리 갈색 격자 수는 $2w + 2h - 4 = \text{brown}$ 이고, 내측 노란색 격자 수는 $(w - 2) * (h - 2) = \text{yellow}$ 를 이룸
- 순회 경계 설정: 노란색 타일이 내부에 최소 1칸 존재하려면 세로 h 는 최소 3 이상이어야 함. h 를 3 부터 전체 면적($\text{total} = \text{brown} + \text{yellow}$)의 제곱근까지만 탐색
- 가로 크기 도출: 전체 면적이 h 로 나누어떨어지는 경우, 가로 길이는 $w = \text{total} / h$ 로 연산됨
- 최종 매칭 판별: 도출된 w 와 h 를 내측 노란색 격자 공식 $(w - 2) * (h - 2) == \text{yellow}$ 에 대입하여 일치하면 즉시 반환

카펫 타일 배치를 만족하는 격자 도출

```
1 // Math.sqrt(total)를 통해 세로(h) 탐색의 최댓값 설정
2 for (int h = 3; h ≤ (int) Math.sqrt(total); h++) {
3     if (total % h == 0) {
4         int w = total / h;
5         // 내측 격자 크기 조건 검증 및 루프 즉시 탈출(break)
6         if ((w - 2) * (h - 2) == yellow) break;
7     }
8 }
```

완전탐색 실행 흐름도



이분탐색

이분탐색의 정의

이분탐색 (Binary Search)

정렬되어 있는 배열이나 리스트에서 탐색 범위를 단계마다 절반씩 좁혀가며 원하는 데이터를 고속으로 찾아내는 기법

- **성능 비약:** 순차 탐색($O(N)$) 대비 극적인 연산량 축소를 보여주며, 복잡도를 $O(\log N)$ 으로 억제함
- **활용 조건:** 반드시 대상 데이터가 오름차순 또는 내림차순으로 사전 정렬되어 있어야만 동작을 보장함
- **매개 변수 탐색 (Parametric Search):** 코딩테스트에서는 단순 값 검색보다, 특정 조건을 만족하는 최댓값/최솟값의 경계를 좁혀 들어가는 최적화 문제 형태로 자주 출제됨

문제 1: 퍼즐 게임 챌린지 (Lv. 2)

- 결정 문제로 변환: "제한시간 내 퍼즐 해결을 보장하는 최소 숙련도"를 구하기 위해, 임의 숙련도 `level` 을 정한 뒤 "이 숙련도로 시간 내 해결 가능한가?(T/F)"로 논리를 전환
- 이분 탐색 적용: 숙련도의 값 범위가 매우 크므로, 탐색 포인터 `left` 와 `right` 를 두어 중간값 `mid` 를 숙련도로 상정하고 이분 검색
- 범위 최적화 규칙:
 - 제한시간 내 해결에 성공하면, 더 낮고 효율적인 숙련도를 테스트하기 위해 경계를 왼쪽으로 축소 (`right = mid - 1`)
 - 실패하여 시간이 초과된다면, 더 높은 숙련도가 요구되므로 경계를 오른쪽으로 확장 (`left = mid + 1`)

결정 문제를 해결하는 이분 탐색

```
1  while (left ≤ right) {
2      long mid = left + (right - left) / 2;
3
4      // 조건 충족 검증 메서드(canSolve)에 따라 이분 탐색 경계 조정
5      if (canSolve(..., mid)) {
6          right = mid - 1; // 만족할 경우 하한선 탐색
7      } else {
8          left = mid + 1;  // 불만족할 경우 요구치 상향
9      }
10 }
```

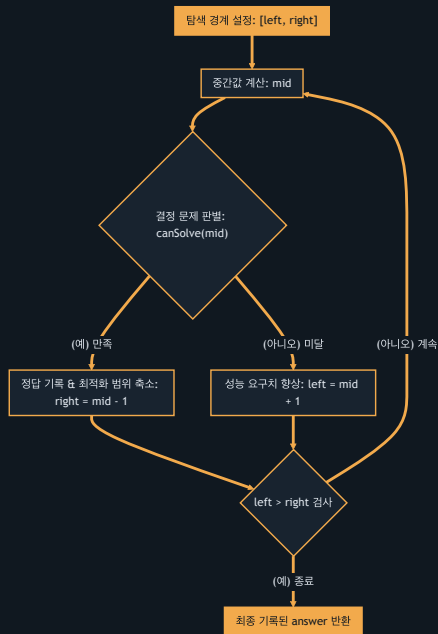
문제 2: 입국심사 (Lv. 3)

- 매개 변수 탐색 설계: "모든 사람이 심사를 마칠 수 있는 최선의 시간"을 직접 탐색하기 어려우므로, "특정 시간 `Time` 내에 N 명 이상 처리 가능한가?"로 결정화
- 이분 탐색 범위 세팅:
 - `left`: 최소 시간 1분
 - `right`: 가장 느린 심사관이 모든 사람을 전부 심사하는 극단적인 최악의 시간 (`(long) n * 가장 느린 심사 시간`)
- 사람 수 합산 검증: 중간 시간 `mid` 동안 모든 심사관이 처리 가능한 사람 수(`mid / 심사 시간`)를 구해 합산한 뒤 N 과 비교
- 자료형 예방: 조 단위 연산 시 오버플로우가 100% 발생하므로, 핵심 변수(`left`, `right`, `mid`, `sum`)는 반드시 `long` 타입을 사용

심사관별 시간 최적화 이분 탐색

```
1  while (left ≤ right) {
2      long mid = left + (right - left) / 2;
3
4      // 주어진 중간 시간(mid) 동안 심사 가능한 총 인원 계산
5      for (int time : times) sum += mid / time;
6
7      // 목표 인원(n) 도달 여부에 따라 탐색 경계 갱신
8      if (sum ≥ n) right = mid - 1;
9      else left = mid + 1;
10 }
```

매개 변수 탐색(Parametric Search) 흐름도



문자열

문자열 제어와 성능

문자열 (String)

텍스트 형태의 데이터를 목적에 맞게 파싱, 치환, 추출, 조립하여 원하는 규격으로 가공하는 핵심 제어 기술

- **Java String 불변성 (Immutable):** String 인스턴스는 한 번 생성되면 내부 문자 배열 리터럴을 직접 훼손할 수 없음
- **메모리 양산 문제:** 문자열 치환(`replace()`), 부분 추출(`substring()`), 더하기 연산 시마다 매번 힙 메모리에 새로운 힙 객체가 별도 생성됨
- **최적화 대책:** 대용량 루프를 도는 문자열 변경 연산에서는 가변 버퍼를 제공하는 `StringBuilder`를 필수 사용해야 가비지 컬렉터(GC) 병목을 회피 가능

문제 1: 숫자 문자열과 영단어 (Lv. 1)

- 1:1 대조 매핑법: 영단어 배열 `"zero"` 부터 `"nine"` 까지 순서대로 배치하여 해당 배열 인덱스(`i`)가 곧 치환해야 할 정수 숫자가 되도록 설계
- 순차 치환 흐름: 0부터 9까지 루프를 수행하며 자바의 `replace(치환대상, 타겟)` API 호출
- 불변성 대처: `replace()` 는 기존 객체 본문 내용을 교체하지 않고 변환 완료된 새 인스턴스를 반환하므로, 연속적으로 원본 변수에 재대입(`s = s.replace(...)`)해야 변경 내역이 축적됨
- 시간 복잡도: 영단어 배열의 크기가 `10` 으로 고정되어 있어 극도의 고속 처리가 가능

영단어 숫자의 정수 일괄 치환

```
1 // String.replace() 메서드를 이용해 영단어와 매핑 정수 치환
2 for (int i = 0; i < words.length; i++) {
3     s = s.replace(words[i], Integer.toString(i));
4 }
5 // Integer.parseInt()를 통한 최종 정수 변환
```

문제 2: JadenCase 문자열 만들기 (Lv. 2)

- 구분자 자르기의 맹점: 단순 공백으로 단어를 자르면 연속된 공백문자가 완전히 소실되어 유실되므로, 문자열 전체를 한 글자 단위(토큰)로 쪼개어 세밀히 정방향 순회해야 함
- 상태 제어 플래그: 단어의 시작 문자 여부를 판별하기 위해 Boolean 플래그(`isFirst = true`) 관리
- 대소문자 전환 규칙:
 - 플래그가 참이면 첫 글자 대문자 전환 (`toUpperCase()`)
 - 플래그가 거짓이면 일반 글자 소문자 전환 (`toLowerCase()`)
 - 현재 순회 중인 글자가 공백(" ")인 경우, 다음 차례 글자는 단어의 첫 글자가 됨 (`isFirst = true` 세팅)

StringBuilder를 이용한 대소문자 순차 변환

```
1 // str.split("")을 통해 한 글자씩 순회하며 상태(isFirst) 검사
2 for (String s : str.split("")) {
3     // 첫 글자 여부에 따라 toUpperCase() / toLowerCase() 적용
4     sb.append(isFirst ? s.toUpperCase() : s.toLowerCase());
5
6     isFirst = s.equals(" "); // 다음 단어 감지
7 }
```

String 메모리 최적화 원리

StringBuilder (가변성)



String (불변성)

